ECE 8920 Final Project: Multi-depot Vehicle Routing Problem with Per-vehicle Weights

Nicholas West

Background

The Vehicle-Routing Problem (VRP) is a well-known problem at the intersection of graph theory, traffic engineering, and combinatorial optimization that seeks to find a fixed number of cycles that cover a graph with minimal weight. This has various real-world applications - for instance, suppose you are the manager of some properties that regularly need serviced by some set of employees. These employees all take off from their own homes in their own vehicles, and have different hourly wages as well as fuel economies. You may seek to minimize the amount of money spent sending your employees to your various properties. This is a kind of VRP, specifically a Multi-Depot Vehicle Routing Problem (MDVRP) with different graph weights per vehicle. Note that the VRP and its variants are very computationally hard to solve, being a superset of the Traveling Salesman Problem (TSP), a well-known NP-Hard problem in the realm of computing.

Three methods are presented in this project: one to solve the MDVRP base case, one to solve the MDVRP with per-vehicle weights, and one to MDVRP with per-vehicle weights on a graph with potentially millions of vertices/edges. These techniques are applied to a small graph of Washington state (derived from Homework 3) and a large graph of Knox County, Tennessee (derived from [1]). After solving, the real-world implications of the methods shown are analyzed.

Method

Easy Problem – MDVRP

The mixed-integer linear programming (LP) formulation of the TSP by Dantzig, Fulkerson, and Johnson is extended in [2] to apply to the VRP. This formulation, in turn, is trivially extended to apply to the MDVRP.

$$G=(V,E)$$

$D \subseteq V$	:=	Depot vertices
D^*	:=	Set of vehicles (note that $\lvert D^* \rvert \geq \lvert D \rvert$)
$x_{i,j}$	:=	Binary variable, or not $i \rightarrow j \in E$ is part of the solution
$x_{i,j}^{(k)}$	:=	Binary variable, or not $i \rightarrow j \in E$ traversed by vehicle k is part of the solution
$c_{i,j}$	:=	Weight of edge $i \rightarrow j$
d_i	:=	Number of vehicles at depot $i \in D$

Figure 1: Symbol definitions

$$z = \min \sum_{i,j \in V} c_{i,j} x_{i,j}$$

$$\begin{array}{llcl}
(1) & \forall i \in V/D & \sum_{j \sim i} x_{i,j} & = & 1 \\
(2) & \forall j \in V/D & \sum_{i \sim j} x_{i,j} & = & 1 \\
(3) & \forall i \in D & \sum_{j \sim i} x_{i,j} & = & d_i \\
(4) & \forall j \in D & \sum_{i \sim j} x_{i,j} & = & d_i \\
(5) & \forall S \subseteq V/D \text{ s.t } S \neq \emptyset & \sum_{j \in S} \sum_{i \notin S, i \sim j} x_{i,j} & \geq & 1
\end{array}$$

Figure 2: LP formulation of MDVRP

Figure 1 defines some symbols used in the following setups, and Figure 2 shows a mixed-integer LP formulation of the MDVRP. Constraints (1) and (2) ensure that each customer vertex is visited exactly once, while (3) and (4) ensure d_i vehicles depart from and return to each depot. (5), the "capacity cut" constraint, ensures that each subgraph formed from the cut of a set of customer vertices is connected to the inverse subgraph, forcing connected cycles given the other constraints. Note that (5) iterates over the power set of V / D – Though the LP is unimodular and does not require the branch-and-cut method, the algorithm still grows in $O(2^n)$ time.

Harder Problem – MDVRP w/ per-vehicle weights

Solving the MDVRP with per-vehicle weights is trickier. Treating each edge's binary constraint as the sum of its per-vehicle binary constraints does not work, as the minimum is simply chosen from the various minima of the edge weights. We must introduce some binary decision variables to force each customer vertex to be serviced by exactly one vehicle.

$$z = \min \sum_{i,j \in V} \sum_{k \in D^*} c_{i,j} x_{i,j}^{(k)}$$

$$\begin{array}{llcl}
(1) & \forall i \in V/D, k \in D^* & \sum_{j \sim i} x_{i,j}^{(k)} & = & y_i^{(k)} \\
(2) & \forall j \in V/D, k \in D^* & \sum_{i \sim j} x_{i,j}^{(k)} & = & y_j^{(k)} \\
(3) & \forall i \in V/D & \sum_{k \in D^*} y_i^{(k)} & = & 1 \\
(4) & \forall k \in D^*, i = \text{depot of } k & \sum_{j \sim i} x_{i,j}^{(k)} & = & 1 \\
(5) & \forall k \in D^*, j = \text{depot of } k & \sum_{i \sim j} x_{i,j}^{(k)} & = & 1 \\
(6) & \forall k \in D^* & \sum_{i \in D, i \neq \text{depot of } k} \sum_{j \sim i} x_{i,j}^{(k)} & = & 0 \\
(7) & \forall k \in D^* & \sum_{j \in D, j \neq \text{depot of } k} \sum_{i \sim j} x_{i,j}^{(k)} & = & 0 \\
(8) & \forall S \subseteq V/D \text{ s.t } S \neq \emptyset & \sum_{j \in S} \sum_{i \notin S, i \sim j} \sum_{k \in D^*} x_{i,j}^{(k)} & \geq & 1
\end{array}$$

Figure 3: LP formulation of MDVRP with per-vehicle weights

Figure 3 shows a modified version of the LP formulation shown in Figure 2 that considers per-vehicle weights. Constraints (1) and (2) are almost identical, except they bind the result to a newly-introduced binary variable, while (3) causes these binary variables to ensure that only one specific vehicle visits a specific vertex. (4), (5), (6), and (7) are extensions of the depot constraints that ensure a vehicle only leaves and departs from its

assigned depot, with its specific edge weights. (8) is almost identical to the “capacity cut” constraint, just with an added summation of all vehicles’ decision variables.

Very Hard Problem – MDVRP w/ per-vehicle weights on a real road network

Both of these formulations can account for large networks where only a subset of vertices are depots/customers by removing their relevant constraints from the customer visitation constraint and removing subsets that only contain such vertices from the “capacity cut” constraint. However, doing so is not practical – a graph with a million vertices would still have on the order of 2 to the power of millions of vertices worth of constraints. As most “real-world” graphs are highly sparse and have millions of vertices, this is not acceptable.

We can reduce the problem domain by, instead of considering the entire graph, we consider a complete graph $K_{|U|}$, where $U \subseteq V$ is the set of customer/depot vertices. Each vertex in $K_{|U|}$ corresponds to a vertex in U, and each edge corresponds to the shortest path between those two vertices in the original graph. This is convenient because the shortest path is both a) unchanging, irrespective of where the vehicle comes from or is going to, and b) can be found efficiently with the A* search algorithm. We can then apply either of the LP formulations presented in Figures 2 and 3 to $K_{|U|}$.

Results

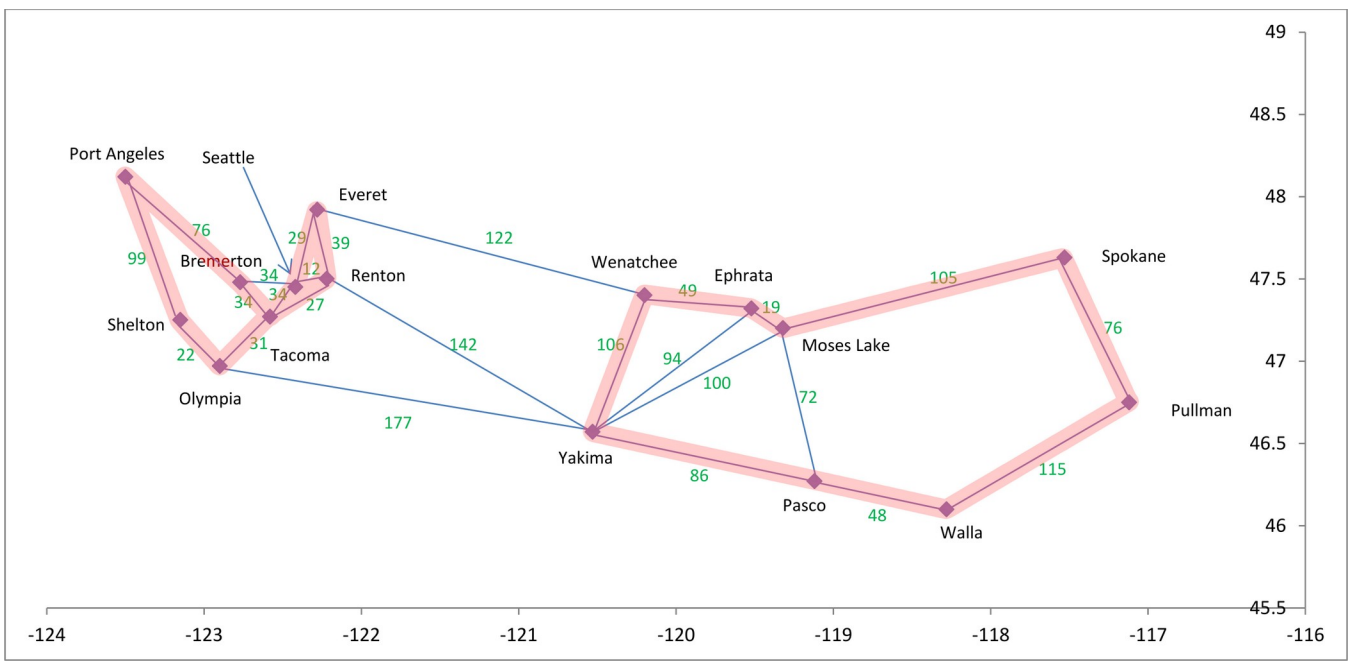

Figure 4: MDVRP solved for small example

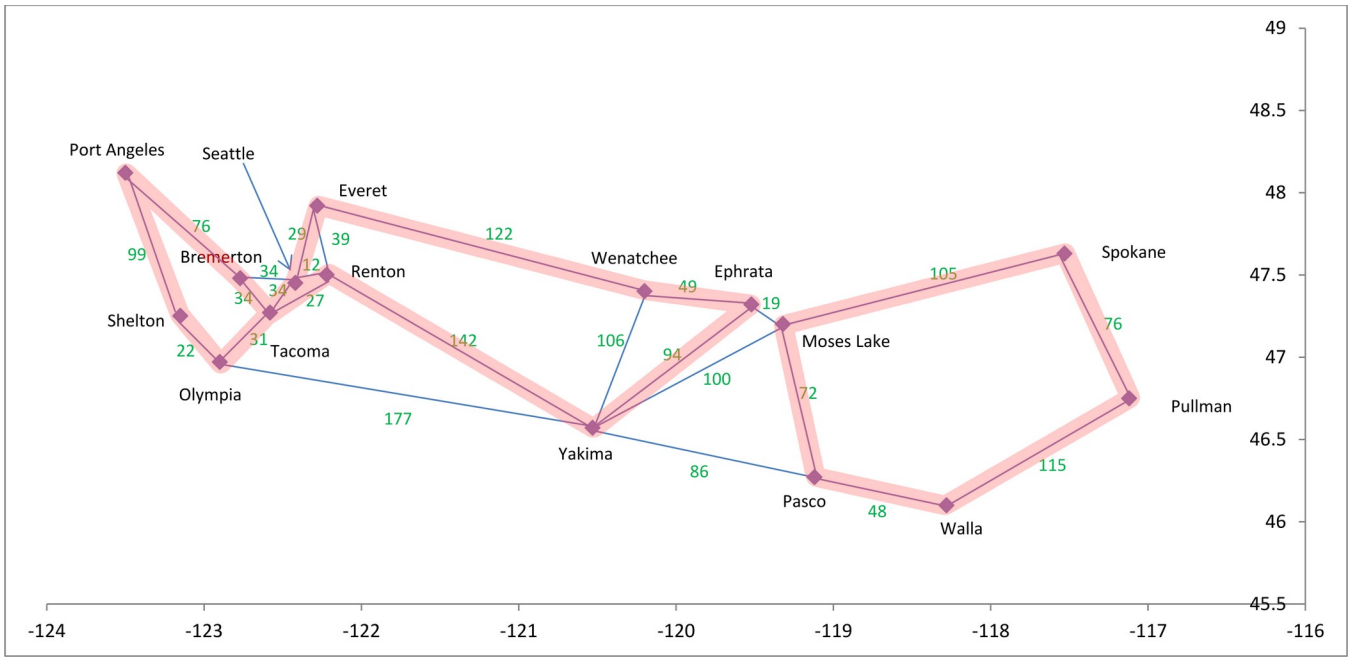

Figure 5: MDVRP with per-vehicle weights solved for small example

Figure 4 shows the result of solving the MDVRP using the first method to the Washington state graph, with two vehicles in Tacoma and one in Pullman. Figure 5 shows the result of applying the second method to the same problem, with the following biases introduced into each vehicle:

- For the first driver from Tacoma, multiply the edge weight by ½ if the destination vertex is north of Tacoma
- For the second driver from Tacoma, multiply the edge weight by ½ if the destination vertex is south of Tacoma
- For the driver from Pullman, don't go to vertices west of Moses Lake (add $M >> 0$ to edges that do so)

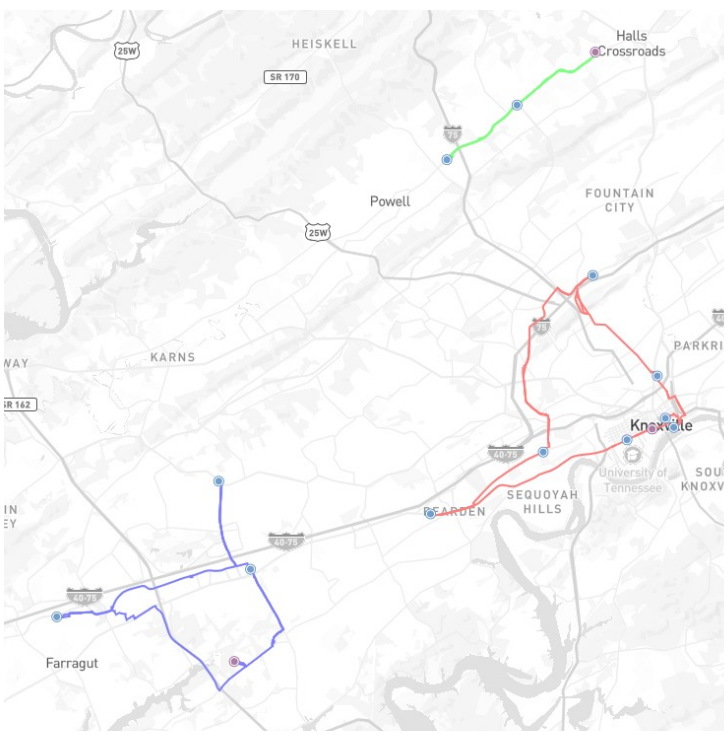

Figure 6: MDVRP solved for large example

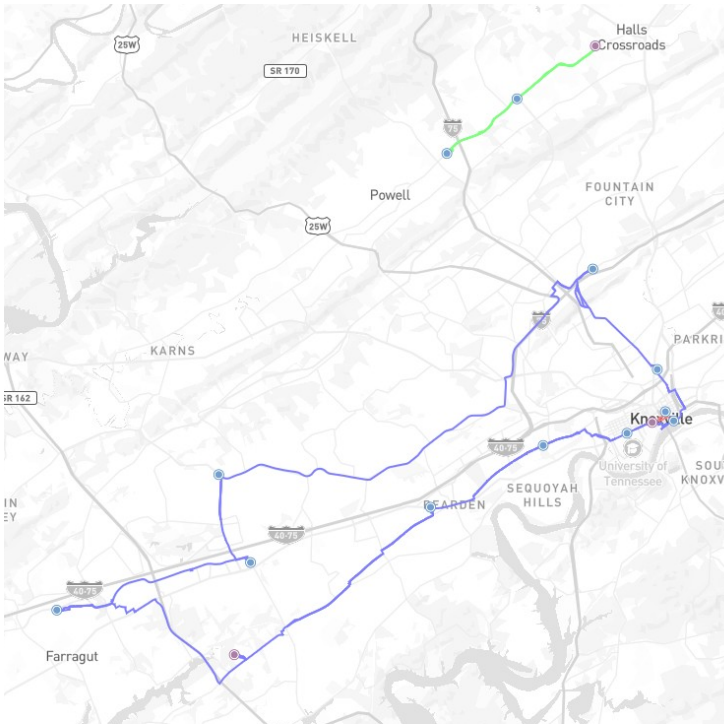

Figure 7: MDVRP with per-vehicle weights solved for large example

Figure 6 shows the result of applying the third method to the graph of Knox County, Tennessee, with twelve customers and three drivers chosen arbitrarily, and all drivers with equal weights. Figure 7 shows the same result, but with a bias introduced to the red vehicle to make it 50% less efficient than all the other vehicles.

Analysis

There are a few practical considerations that should be made before applying these methods to “real-world” routing problems. When I was formulating these problems I noticed a lack of discussion in the literature about per-vehicle MDVRPs, and I suspect that is because such problems tend to lead to either the same result as a typical MDVRP, or they lead to one vehicle’s costs dominating all the others and doing all the work. In particular, if two vehicles differ linearly in their weights, then either said difference won’t be enough to justify a different solution compared to a traditional MDVRP, or, if it ever becomes the case that one customer is better serviced by a different vehicle than the one in the traditional MDVRP, it will also service all the other customers that the other vehicle would’ve serviced, since it is already within reach of the other vehicle’s old cycle and its weights are uniformly lower.

If we are concerned about this, one potential solution may be a time-constrained Lagrangean relaxation, similar to the one performed in Homework 4. We’d need to be careful to ensure that we only consider the maximum time that any one vehicle is on the road, instead of the total time spent by all vehicles. Nonetheless, this may be a desired method to ensure that all vehicles are doing a substantial amount of work, even if the nominal cost would be lower in the case of one vehicle doing all the work. On the other hand, if we’re not concerned with burdening one vehicle with all the work, we may simply reformulate the LP so that the depot constraints become a “≤” constraint instead of a “=” one. This would make it so a vehicle *may* depart from/return to a depot without requiring it to.

When I was running the code for the third method, the most time-consuming part by far was performing all the A* searches to put together the complete graph. A* is efficient, but the original graph was so large that it was still very time-consuming. There are a number of things that can be done to speed up this step. The first, which I

did, was applying parallelism – since every edge weight can be computed independently of the other, this is, in academic terms, an “embarrassingly parallel” problem. The second optimization would be to use a simpler heuristic function. In the code, I used great-circle distance, which is both accurate and consistent. However, a simple Euclidean distance, while not consistent across *large* distances across Earth’s surface, probably would’ve been just fine within the bounds of Knox County, and more importantly much simpler to compute than the Haversine formula used for great-circle distance. There are many other preprocessing and optimization steps specific to geospatial data that “real-world” map routing software such as [3] performs to give sub-second shortest-path results.

Finally, although the reduction to $K_{|U|}$ does allow us to solve the MDVRP for otherwise computationally-infeasible graphs, it still requires the “capacity cut” constraint that forces our solution to ultimately have an $O(2^n)$ time/space complexity. There is much discussion in the literature of heuristic and probabilistic approximations to the VRP and its subproblems, as seen in [4], [5], and [6]. Extending one of these methods to apply to the MDVRP with per-vehicle weights will be of significant interest for future work.

References

[1] Various OpenStreetMap contributors. “tennessee-latest.osm.pbf.” Distributed by Geofabrik GmbH Karlsruhe. https://download.geofabrik.de/north-america/us/tennessee.html (accessed Apr 23, 2026).

[2] P. Toth and D. Vigo, The Vehicle Routing Problem, in SIAM Monographs on Discrete Mathematics and Applications, Philadelphia, PA, USA: Society for Industrial and Applied Mathematics, 2002, doi:10.1137/1.9780898718515.

[3] GraphHopper GmbH, "GraphHopper," ver. 11.0, Open Source, Oct 2025, [Online]. Available: https://github.com/graphhopper/graphhopper/releases/tag/11.0.

[4] Wang, Chun Bo, and Jun Guo, “A New Hybrid Algorithm Based on Artificial Fish Swarm Algorithm and Genetic Algorithm for VRP.” Applied Mechanics and Materials, vol. 325–326, pp. 1722–25, Jun 2013, doi:10.4028/www.scientific.net/AMM.325-326.1722.

[5] Lin, Shih-Wei, et al,“Solving the Truck and Trailer Routing Problem Based on a Simulated Annealing Heuristic.” Computers & Operations Research, vol. 36, no. 5, pp. 1683–92, May 2009, doi:10.1016/j.cor.2008.04.005.

[6] R. Stefan, and D. Pisinger, “A Unified Heuristic for a Large Class of Vehicle Routing Problems with Backhauls.” European Journal of Operational Research, vol. 171, no. 3, pp. 750–75, Jun 2006, doi:10.1016/j.ejor.2004.09.004.